



SIMULATION DE BOIDS

MEMBRES DU GROUPE :

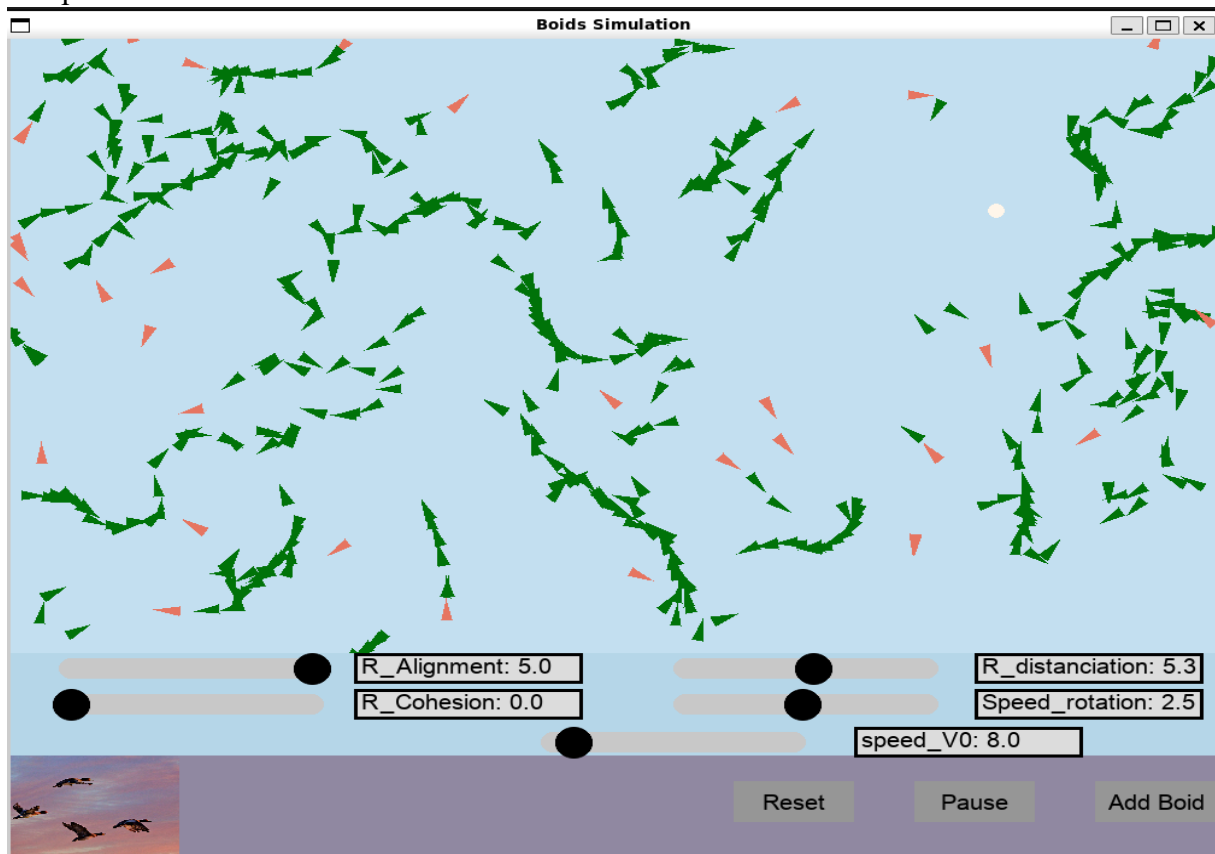
- ❖ NGUETI NGOUADJIO TERRYL
- ❖ NDE FOTSO WILLIAM K.
- ❖ NGOUA ESSONO FRANCOIS

SOMMAIRE

- I. DESCRIPTION DETAILLÉE DU PROGRAMME**
- II. JUSTIFICATION DU CHOIX DE CONCEPTION**
- III. DESCRIPTION DETAILLÉE DES FONCTIONNALITÉS IMPLÉMENTÉES**
 - a- CAS DES FONCTIONNALITÉS DE BASES
 - b- CAS DES FONCTIONNALITÉS AVANCÉES (extensions)
- IV. DIFFICULTÉS RENCONTRÉES**

I. DESCRIPTION DÉTAILLÉE DU PROGRAMME

Notre programme informatique BOID basé sur un programme informatique de vie artificielle développé par **Craig W. Reynold**, implémente une simulation interactive d'une nuée d'oiseaux en vol, utilisant Pygame pour l'interface graphique et Numpy pour les calculs mathématiques. L'acronyme BOID est une contraction de Bird-oid (qui a la forme d'un oiseau). Les BOIDS, représentés par des triangles pleins noirs, se déplaçant dans un espace bidimensionnel avec des bords toriques, adoptent des comportements collectifs réalistes basés sur trois règles principales : **alignement** (si un voisin est dans cette zone, le BOID ajuste sa direction pour s'aligner avec lui), **cohésion** (si un voisin est dans cette zone, le BOID se rapproche du centre de ce groupe pour rester avec lui) et **distanciation** (si un voisin entre dans cette zone, le BOID s'en éloigne pour éviter une collision), influencées par les paramètres ajustables via des **SLIDERS**. L'interface comprend également des boutons permettant de mettre la simulation en pause, d'ajouter un BOID ou de réinitialiser la nuée avec des options spécifiques. Un prédateur, ajouté pour perturber les BOIDS, se déplace indépendamment et provoque des comportements d'évasion chez ceux-ci. La simulation utilise une grille spatiale pour optimiser la recherche des voisins et réduire la complexité des calculs. Les interactions utilisateur, comme la suppression de BOIDS ou la réinitialisation par clic, sont gérées dans des threads pour préserver la fluidité de l'animation. Notre programme offre également, une personnalisation des paramètres, tels que la vitesse des BOIDS ou les forces des comportements, et affiche les messages contextuels pour informer l'utilisateur. Il combine des fonctionnalités visuelles dynamiques avec une structure de code modulaire et extensible, permettant une exploration approfondie des comportements collectifs.



II. JUSTIFICATION DU CHOIX DE CONCEPTION

Nous avons choisi de concevoir notre programme en **Python** qui simule une nuée de BOIDS, en utilisant **Pygame** comme bibliothèque principale pour l'interface graphique. Ce choix repose sur plusieurs considérations liées à la simplicité d'utilisation, aux fonctionnalités requises, et à l'objectif de rendre la simulation interactive et visuellement explicite.

- ✚ Le langage **Python** est connu pour sa syntaxe simple et intuitive, ce qui permet de se concentrer sur les concepts de simulation et d'interaction sans être ralenti par la complexité du langage.
- ✚ Les bibliothèques disponibles, comme **Pygame**, **numpy**, et **pygame_widgets**, facilitent le développement rapide d'applications interactives, incluant les calculs mathématiques et les interfaces graphiques.

III. DESCRIPTION DETAILLÉE DES FONCTIONNALITÉS IMPLÉMENTÉES

Notre programme implémente deux catégories de fonctionnalités.

a. CAS DES FONCTIONNALITÉS DE BASE

Ajout progressif de BOIDS :

Les BOIDS sont ajoutés dans un espace bidimensionnel avec des bords toriques, de manière progressive et aléatoire avec une boucle **FOR**.

Mise en pause et reprise :

Notre programme offre à l'utilisateur la possibilité de mettre la simulation de BOIDS en pause. Cette fonctionnalité est liée à un bouton ***Pause/Resume*** intégré à l'interface de simulation. Une variable `is_not_pause` a été créée et initialisée à `false` dans le programme.

Lorsqu'un utilisateur clique sur ce bouton :

- Si la simulation était en cours (BOIDS en vol), les BOIDS cessent de bouger mais restent affichés, et la variable `is_not_pause` prend la valeur `true`.
- Si la simulation était en pause, les BOIDS reprennent leur mouvement (vol).

Remise à zéro :

Elle permet à l'utilisateur via un clic sur le bouton ***Reset*** intégré à l'interface de simulation, de réinitialiser l'état actuel de la simulation de BOIDS. Deux options sont offertes :

- **Supprimer tous les BOIDS** (remettre à zéro la simulation sans aucun BOID).
- **Réinitialiser avec un nombre spécifique de BOIDS** (générer un nouvel ensemble de BOIDS selon le choix de l'utilisateur).

Cette interaction qu'on a intégré à la simulation, améliore l'expérience utilisateur en permettant une personnalisation rapide (expliquée en détaille plus bas) de la simulation sans devoir redémarrer le programme.

Paramètres ajustables :

Ils permettent de modifier en temps réel les comportements et les caractéristiques de la simulation. Ces réglages au moyen des **SLIDERS** (fonction de Pygame), offrent à l'utilisateur un contrôle interactif pour explorer différentes dynamiques des BOIDS et personnaliser l'expérience de simulation.

- La vitesse de déplacement des BOIDS (**speed_v0**) : Lorsque l'utilisateur déplace le SLIDER (cercle noir) **speed_v0** vers la droite (augmentation de la valeur actuelle), la vitesse s'élève et cela rend les BOIDS plus dynamiques et rapides dans leurs déplacements. S'il le déplace vers la gauche (abaissement de la valeur actuelle), la vitesse baisse et cela rend les BOIDS moins rapides dans leurs déplacements. Ainsi, on peut observer en détail leur comportement.

- Le rayon de la zone d'Alignement (**R_alignement**) : Lorsque l'utilisateur déplace le SLIDER **R_alignement** vers la droite, cela provoque un alignement plus strict des BOIDS, favorisant des formations de vol en groupe. S'il le déplace vers la gauche, cela diminuerait cet effet, rendant les BOIDS plus indépendants.
- Le rayon de la zone de cohésion (**R_cohesion**) : Lorsque l'utilisateur déplace le SLIDER **R_cohesion** vers la droite, cela entraîne un rassemblement des boids en groupes serrés. S'il le déplace vers la gauche, cela entraînerait une dispersion des BOIDS, créant des mouvements plus isolés.
- Le rayon de la zone de distanciation (**R_distanciation**) : Lorsque l'utilisateur déplace le SLIDER **R_distanciation** vers la droite, cela éloigne les BOIDS les uns des autres, créant des formations plus lâches. S'il le déplace vers la gauche, cela entraînerait des regroupements plus denses, mais peut provoquer des collisions visuelles.
- La vitesse de rotation maximale des BOIDS (**Speed_rotation**) : le SLIDER **Speed_rotation**, contrôle la vitesse angulaire maximale à laquelle un BOID peut tourner pour se diriger vers un objectif, éviter un obstacle, ou respecter les règles de cohésion, d'alignement et de séparation. Lorsque l'utilisateur le déplace vers la droite, les BOIDS réagissent rapidement aux changements de direction. S'il le déplace vers la gauche, les BOIDS prendraient plus de temps pour ajuster leur trajectoire.

Changement de couleur en fonction des interactions :

Dans cette simulation, les BOIDS changent dynamiquement de couleur pour refléter les forces qui influencent leur comportement.

- **Vert** pour la **distanciation** : Les BOIDS se colorent en vert lorsqu'ils s'éloignent les uns des autres pour éviter une trop grande proximité.
- **Rouge** pour l'**alignement** : Les BOIDS se colorent en rouge lorsqu'ils ajustent leur orientation pour s'aligner avec leurs voisins proches.
- **Bleu** pour la **cohésion** : Les BOIDS se colorent en bleu lorsqu'ils se rapprochent pour former des groupes ou des formations.

b. CAS DES FONCTIONNALITÉS AVANCÉES (extensions)

Ajout de BOIDS via le bouton nommé « Add Boid », suppression de BOIDS via la souris et personnalisation du « Reset » :

- **Ajout de boids** via le bouton nommé « *Add Boid* » : La fonction “add_boid()” est liée au bouton “**Add Boid**”. Lorsqu'on clique dessus, un nouveau boid est créé et ajouté à la liste principale. L'ajout est confirmé par un message affiché via la fonction “display_message()”.

- **Suppression de boids via la souris** : Quand on clique dans la fenêtre de simulation, la fonction "check_boid_click()" vérifie si un boid a été sélectionné en comparant la position du curseur avec celle des boids. Si un boid est suffisamment proche du clic (distance < 10 pixels), il est ajouté à une liste temporaire « to_remove ». Les boids de cette liste sont ensuite supprimés de la liste principale, et un message est affiché avec le nombre de boids restants.
- **Personnalisation** du « *Reset* » : La réinitialisation est déclenchée via le bouton "**Reset**", qui appelle la fonction "reset_in_thread()". Cette fonction s'exécute dans un thread séparé pour éviter de bloquer l'interface utilisateur. Lors de la réinitialisation, deux options sont proposées à l'utilisateur via une boîte de dialogue :
 - **Supprimer tous les boids** : La liste flock (liste principale) est vidée.
 - **Créer un nombre spécifique de boids** : L'utilisateur saisit un nombre entre 1 et 10000. Si une valeur valide est entrée, tous les boids actuels sont supprimés, puis un nombre spécifié de nouveaux boids est ajouté.

Après l'opération, un message informatif est affiché.

Boid ennemi (prédateur) :

La simulation intègre un prédateur, un objet qui hérite de la classe Boid et interagit directement avec les BOIDS via la fonction avoid. Ce prédateur influence leur comportement en déclenchant une réaction d'évitement lorsqu'il s'approche. Les BOIDS, tout en respectant leurs règles de cohésion, séparation et alignement, adaptent leur trajectoire pour fuir le prédateur. L'utilisateur peut ajouter ou retirer des prédateurs dynamiquement grâce aux touches Espace (ajout) et D (suppression), enrichissant l'interactivité et la complexité de la simulation.

Optimisation des performances :

- **Division de l'espace en grille spatiale** :

L'espace de simulation est subdivisé en **une grille 2D** composée de cellules de taille fixe (20px * 20px). Chaque cellule stocke les BOIDS qui s'y trouvent. Cela réduit le coût de recherche des voisins en limitant les calculs aux BOIDS présents dans la cellule actuelle et ses cellules adjacentes (9 cellules maximum au total), au lieu de considérer tous les BOIDS.

- **Traitement localisé des comportements** :

Les règles d'alignement, de cohésion et de séparation sont calculées uniquement à partir des voisins proches obtenus via la grille, ce qui optimise les calculs tout en conservant un comportement réaliste.

- **Réinitialisation rapide de la grille :**

Avant chaque mise à jour, la grille est effacée efficacement en vidant simplement les listes de chaque cellule avec la fonction avec la fonction **clear** de la classe **SpatialGrid**. Les BOIDS sont ensuite réattribués aux cellules en fonction de leur position actuelle.

- **Réduction du nombre de comparaisons :**

Lors de la mise à jour des interactions, les voisins d'un BOID sont récupérés directement à partir de la grille avec la fonction **get_neighbors** de la classe **SpatialGrid**. Cela diminue significativement les opérations de vérification des distances, particulièrement dans des simulations avec un grand nombre de BOIDS.

IV. DIFFICULTÉS RENCONTRÉES

- ✚ Gérer les comportements (alignement, cohésion, séparation, gestion du prédateur) tout en maintenant une simulation réaliste a représenté un défi conceptuel.
- ✚ Lors de l'ajout du prédateur à la simulation, certaines difficultés ont émergé, notamment dans la gestion des différentes forces agissant sur les BOIDS. Il a fallu ajuster l'équilibre entre les forces d'évitement, de cohésion, de séparation et d'alignement pour garantir un comportement réaliste. Intégrer le prédateur tout en maintenant une fluidité et une cohérence dans les mouvements des BOIDS a constitué un véritable défi, demandant plusieurs ajustements pour éviter des conflits ou des comportements incohérents.
- ✚ Intégrer les sliders pour moduler dynamiquement les comportements des boids (alignement, cohésion, séparation) et les appliquer correctement à chaque boid a demandé une gestion précise des paramètres.